

Succinct Filters for Sets of Unknown Sizes

论文 Review

刘威 (A) 张书钺 (B) 陈戚 (C)

评述论文: Liu, Yin and Yu, ICALP 2020

最终提交版 · 2026 年 7 月

1. 摘要

本文研究最终集合规模或紧容量上界事先未知时的动态 approximate membership。其难点在于，filter 为了节省空间已经丢弃了枚举原 key 所需的信息，因而不能像 exact dictionary 那样直接重建扩容。Liu-Yin-Yu 通过变长哈希前缀、prefix matching、短串 truth table、长串 succinct 结构和两代去均摊迁移，在 $n=\omega(\log u)$ 、 $n < u$ 时使用 $n(\log(1/\epsilon) + \log \log n + O(\log \log \log u)) + u^c$ 位，并在 no-failure 执行中给出插入和查询的 worst-case $O(1)$ 。对任意插入序列，整个过程曾报告 failure 的概率至多为 $\delta = u^{-C}$ ；无 failure 时无漏报，非成员误报率至多为 ϵ 。本 Review 详细解释这一结果与 PSW 信息论下界的关系，重建其实现、正确性、误报、failure、时间和空间论证的依赖链，并公开保留阶段下标、字面常数 10 和位级迁移峰值等 Q1/Q3 核查项。

2. 引言

成员查询出现在缓存、存储、网络和数据库等诸多系统中。Exact dictionary 保留足够的 key 信息以回答精确查询；filter 允许将少量非成员错认为成员，但对已插入元素不允许漏报，因而能用更少比特完成预过滤。

当容量上界 N 已知时，结构可以在初始化为 N 个 key 配置空间。但在一个持续增长的系统中，紧的最终上界常常无法事先得到：高估 N 会在 $n \ll N$ 时浪费空间，低估又会触发扩容。对 dictionary，扩容可通过枚举原 key 重建；对 filter，内部状态未必足以恢复原 key，因而这个看似平常的操作成为理论难点。

论文的核心问题是：能否在没有紧最终容量上界时，让空间始终依赖当前规模 n ，同时保持无漏报、误报率至多 ϵ ，并把插入和查询都做到高概率意义下的最坏常数时间？PSW 已证明 unknown-size 设定必须支付额外 $n \log \log n$ 信息，并给出接近下界的构造；LYY 的任务是在收紧该项领先常数的同时，解决去均摊扩容和位级 succinctness 之间的冲突。

下文先给出问题与结果 (A)，再述相关工作 (C)，然后展开构造与证明 (B)，最后评价与后续 (C)。

3. 问题定义和研究意义

全局符号（全文统一；log 以 2 为底）

符号	含义
$[u]$	全集 $\{0, \dots, u-1\}$
S	当前逻辑集合 $S \subseteq [u]$
n	当前插入规模；本文默认互异 key 序列
ϵ	误报率（false positive rate）
$\delta = u^{-C}$	曾报告 failure 的概率上界
N	传统预置容量上界（本文希望摆脱对紧 N 的依赖）
w	word-RAM 字长，Theorem 10 中 $w = \Theta(\log u)$
c, C	预计算空间和 failure 指数的常数， $0 < c < 1$ 、 $C > 1$

$\text{insert}(x)$ 将 key 加入 S ； $\text{lookup}(x)$ 回答成员性。Dictionary 不允许误报或漏报；filter 对 $x \in S$ 必须回答 YES，对 $x \notin S$ 允许以至多 ϵ 的概率回答 YES。“Unknown size”指初始化时不知道最终规模或紧的 N ，不是 u 未知，也不是结构不知道当前 n 。

已知容量 filter 的主信息论成本为 $N \log(1/\epsilon)$ 位。Unknown-size 模型要求每个中间时刻的空间都跟随当前 n ，PSW 下界证明除误报信息外还需支付近似 $n \log \log n$ 的额外信息。这使 LYY 的目标不是单纯“会扩容”，而是在这一不可避免的空间尺度上进一步实现 worst-case $O(1)$ 操作。

概率口径： ϵ 是 no-failure 条件下的非成员误报率； δ 是任意插入序列上“整个过程曾报告 failure”的序列级事件上界。无漏报和操作时间的对外说明均保留 no-failure 条件。

4. 相关研究

比较相关工作之前，先固定本文的目标组合：事前没有紧的最终容量上界；空间随当前插入规模变化；无漏报、误报率至多 ϵ ；无 failure 时插入和查询为 worst-case $O(1)$ 。因此“动态”“可扩容”和“支持删除”必须拆成空间参数、查询路径、更新时间、删除、工程实现和概率口径分别比较。

4.1 从 Bloom filter 到多组件扩展

Bloom filter 是 approximate membership 的历史起点 [2]；Carter 等人给出更一般的问题与下界基础 [3]。经典 Bloom filter 通常根据目标容量设定位数组和哈希参数；持续超额插入会使误报偏离原目标。

Scalable Bloom Filter (SBF) 在组件达到阈值后追加更大组件，并按几何级数收紧各组件误报预算；查询要逐个测试已有组件 [almeida2007scalable, § 4–§ 5]。Dynamic Bloom Filter (DBF) 也追加组件，其论文给出的平均查询复杂度与组件数相关，误报表达式随组件数增长 [guo2006dynamic, § III]：

query — $\boxtimes$ Filter_0 — $\boxtimes$ Filter_1 — $\boxtimes$... — $\boxtimes$ Filter_s

它们说明未知最终规模可以用工程化组件增长处理，但没有自动获得本文“位级当前规模空间 + 固定常数个查询结构 + 可比最坏时间”的组合保证。本文也不能据此被概括为“全面优于 Bloom”。

4.2 Succinct dictionary 与去均摊背景

Exact dictionary 的稀疏信息论基准接近 $\log \text{binom}(u, n) \approx n \log(u/n) + O(n)$ ；filter 的主基准是 $n \log(1/\epsilon)$ 。编码对象不同，不能直接比较两个对数项的大小。Raman–Rao、Demaine 等工作提供 succinct dynamic dictionary 背景 [raman2003succinct; demaine2006dictionariis]；去均摊 cuckoo hashing 则说明如何把重建拆到多次更新 [6]。LYY 的困难在于同时满足 bit-level succinctness 和 worst-case 去均摊，而不是把任一背景结构原样套用。

Quotient filter 与 Cuckoo filter 是原文未引用、由本 Review 外补的工程对照。前者支持删除、合并和 resize，但依赖槽数、负载率与 cluster 长度 [bender2012quotient, § 3–§ 4]；后者查询两个桶并支持删除，但插入可能 relocation 或失败，保证依赖桶数、桶容量、负载和 fingerprint 长度 [fan2014cuckoo, § 4–§ 5]。删除、局部性和吞吐是有意义的额外维度，却不等于已经证明本文的 unknown-size 空间式。

4.3 直接前作：PSW 2013

Pagh–Segev–Wieder (PSW) 直接研究最终规模未知、空间在各中间规模受约束的 approximate membership [9]。其 Theorem 3.1 的编码下界说明，除 $n \log(1/\epsilon)$ 外还要支付约 $n \log \log n$ 的额外信息；下界不依赖操作时间。PSW 的上界插入为 expected amortized $O(1)$ （按 LYY 转述），且 $\log \log n$ 上界项没有钉死领先常数。

LYY 的推进是组合性的：在相应条件下收紧 $\log \log n$ 领先项，并在 no-failure 执行中同时给插入、查询 worst-case $O(1)$ [1]。可靠说法是“在 insertion-only unknown-size 模型中收紧空间与时间语义”，而不是“首次提出 unknown-size”或无条件“达到全部信息论最优”。

4.4 2020 年后的三类进展

- Aleph Filter (2024) 直接引用并比较 LYY，强调可运行的无限增长、删除和内存—误报率权衡 [dayan2024aleph, § 7]。但其部分 void duplicate 清理按生命周期 amortized 分析 (§ 5.2)，不能直接等同于 LYY 的 no-failure worst-case 口径。
- Kuszmaul–Walzer (2024) 研究支持插入、删除且有容量约束的 fully dynamic filter，证明删除模型需要线性额外空间；Theorem 3.1 在相应区间给出 $>0.35n - o(n)$ 的额外项 [14]。它说明删除改变信息论约束，不是对 LYY 正确性的否定。

- Resizable Retrieval (2026 预印本) 把空间按当前集合大小 n 计, 并导出支持插入/删除的 filter 推论。Corollary 3.14 的空间为 $n \log(1/\epsilon) + O(n \log \log(U/n)) + \text{polylog } U + O(U^\delta)$, 操作为相对当前 n 的 constant time with high probability [15]。它直接承接 PSW/LYY, 但参数和概率口径不同; 截至 2026-07-28 仍只按 arXiv/CoRR 预印本引用。

InfiniFilter 目前为 E2; Li 等人 2023/2024 的 exact dynamic dictionary 工作为 E1。它们保留在研究矩阵展示检索过程, 不进入本稿的确定性跨论文结论。完整时间线和证据等级 与。

5. 论文主要结果

5.1 定理层次

结果	参数与空间	时间、误报与 failure
正式 Theorem 10	$n = \omega(\log u)$, $n < u$; $n(\log(1/\epsilon) + \log \log n + O(\log \log \log u)) + u^c$ 位	每次插入/查询 worst-case $O(1)$; 序列级 failure 至多 $\delta = u^{-\Omega(1)}$; no-failure 时 $FP \leq \epsilon$ 、无 FN
非正式 Theorem 1	$n > u^{\Omega(1)}$ 时简化为 $(1+o(1))n(\log(1/\epsilon) + \log \log n)$	最坏常数时间 with high probability

Theorem 1 的 $n > u^{\Omega(1)}$ 是用来将 $O(\log \log \log u)$ 和 u^c 吸收进 $(1+o(1))$ 的更强展示条件, 不是 Theorem 10 的字面范围。

5.2 PSW 下界与领先项

PSW Theorem 3.1 在一个规模区间内对每元素位数 β 给出参数化下界。证明先固定数据结构的随机性, 再用几何块分解插入序列; 在若干中间时刻中找到一个正回答集合增长很小的块, 并用 Chernoff 界控制该块中预先误报的元素数; 然后把数据结构的中间状态当作编码的一部分, 利用块前后正回答集合压缩该块。如果所有中间状态都太小, 就能把至少 $u^{n/3}$ 条序列编码得比 $n \log u - O(1)$ 的计数下界更短, 导出矛盾。

PSW Theorem 1.1 字面得到 $(1-o(1))n \log(1/\epsilon) + \Omega(n \log \log n)$; 更细的 $(1-O(\epsilon))$ 领先系数来自 Theorem 3.1 的参数选择。因此 LYY 式 (2) 只解释本构造为何出现 $\log \log n$, PSW 编码论证才说明它对 unknown-size 结构普遍不可避免。当 $\epsilon = o(1)$ 时, LYY 上界与 PSW 参数化下界的 $\log \log n$ 领先项对齐; 对固定 ϵ , 不将 $(1-O(\epsilon))$ 扩张为严格系数 1。

6. 核心技术直觉

6.1 总路线

在最终规模未知时, 构造目标是: 空间跟随当前 n , 无漏报, 误报 $\leq \epsilon$, 且在无 failure 时插入/查询为 worst-case $O(1)$ 。

总路线:

1. 全局哈希 h 把 key 映成长比特串, 不保存原始 key。
2. 第 n 次插入只写入 $h(x)$ 的前 $\ell_{i^*(n)}$ 位, 其中 $i^*(n) = \text{ceil}(\log n)$ 。
3. 查询化为 prefix matching: 是否存在已存串 s 为 $h(y)$ 的前缀。
4. 短前缀用真值表 T , 长前缀用已知容量结构 $D(m, \ell)$; 未知规模由「两代 D/T 并存 + 后台迁移」拼装 (Claim 13 $\rightarrow$ Lemma 11 $\rightarrow$ Theorem 10)。

相对 PSW: 真值表处理短串; 去均摊、可查询的迁移; 更紧底层冗余以支撑 $\log \log n$ 项领先常数 (条件见 A)。

6.2 变长前缀与误报预算

论文命题 (§ 3.1 式 (2)):

$$\ell_i = i + \log(1/\epsilon) + \log i + \log \log \log u + 2$$

项	作用
i	随阶段增长加长前缀
$\log(1/\epsilon)$	主误报成本
$\log i$	跨阶段并合预算
$\log \log \log u, +2$	冗余与并合余量

非成员误报经阶段内前缀数与 $2^{\{-\ell_i\}}$ 的 union bound 受控（精算见 A）。迁移只改存放位置，不改变该前缀写入时所按的 ℓ_i 预算。

6.3 四个活跃结构

操作参数 $i=i^*(n)$:

结构	职责
D_i	当前长前缀主写入
$D_{\{i-1\}}$	旧长前缀；迁移源；仍可查
T_i	短串；接收迁移
$T_{\{i-1\}}$	更短层；迁出时扩展为 y_{-0}, y_{-1}

「已初始化」的下一层可能存在，但 Lookup 不查询它们。

开放缺口 (Q1/Q3)：Claim 13 证明段半开区间句与 i^* 下标冲突未形式关闭；正文操作一律跟算法句 i^* 。

小边界： $n=1$ 时无 $D_{\{-1\}}/T_{\{-1\}}$ ；式 (2) 并合自 $i \geq 1$ ， ℓ_0 特判待与 A 统一。

6.4 为何不是“可扩容 Bloom”

工程 scalable/dynamic Bloom 可增长，但通常查询多个组件、空间/时间保证与本文 word-RAM 定理不同 (C)。本文要同时：空间相对当前 n 的 succinct 主项、只查常数结构、whp 最坏 $O(1)$ 。

6.5 贯穿教学示例（唯一）

连续插入 $x_1 \cdots x_8$ （人为示意前缀，非真实哈希，不证明概率/空间）：

n	i^*	新键进入	查询活跃	维护推进
1	0	D_0	D_0, T_0	—
2	1	D_1	D_0, D_1, T_0, T_1	迁 T_0/D_0 ; init T_2/D_2
3-4	2	D_2	D_1, D_2, T_1, T_2	迁 T_1/D_1 ; init T_3/D_3
5-8	3	D_3	D_2, D_3, T_2, T_3	迁 T_2/D_2 ; init T_4/D_4

7. 数据结构详细实现

7.1 算法说明（抽象；非工程代码）

状态： n, h, failed ，各层 $T[j], D[j]$ ，init/destroy 进度。 $i^*(n) = \text{ceil}(\log n)$ ， ℓ_i 如式 (2)， $\text{Pref}(z, L)$ 为前 L 比特。

Lookup(y) ($\neg \text{failed}, n \geq 1$)： $i \leftarrow i^*(n)$ ， $z \leftarrow h(y)$ ；若 $D_{\{i-1\}}/D_i/T_{\{i-1\}}/T_i$ 任一对相应前缀查询命中则 YES，否则 NO。至多四次底层 query。

Insert(x)： $n \leftarrow n+1$ ， $i \leftarrow i^*(n)$ ， $s \leftarrow \text{Pref}(h(x), \ell_i)$ ；若无更短前缀覆盖则 $D.\text{insert}(D_i, s)$ （失败则 $\text{failed} \leftarrow \text{true}$ ）；再执行 10 次维护步（原文）。

一轮维护 (MigrateOneStep(i))，对应 Claim 13:

1. 若 T_{i-1} 非空: decrement 得 y , 则写入 T_i 的 $y \cdot 0$ 与 $y \cdot 1$;
2. 若 D_{i-1} 非空: decrement 得 y , 则 $|y| > i$ 进 D_i , 否则进 T_i ;
3. 旧层已空则推进 destroy;
4. 旧层已销毁则推进 $\text{initialize}(T_{i+1}), \text{initialize}(D_{i+1})$ 。

字面常数 10: 原文每次执行 10 轮; 小组未从黑盒 $O(m)$

隐藏常数独立复算其充分性 (Q3)。渐近上存在足够大的常数轮数。写入层就绪: 单次 Insert 不得 while 到初始化完成; 依赖阶段就绪不变式 + 每插常数轮推进。

7.2 底层 $D(m, \ell)$

黑盒: $\text{initialize/destroy}$ (各 $O(m)$)

次调用完成)、insert、query (是否存在前缀)、decrement。空间约 $q(\ell - \log m + 2 \log \log \log u) + O(m)$ 。写入 D_i 的串来自 core set, 以匹配随机性假设。

§ 5 内部: main table $\rightarrow$ subtable $\rightarrow$ adaptive prefixes / navigator $\rightarrow$ data blocks (动态插入 + 后台重组静态)。位级冗余与式 (3)(4)(5) 精算以 A 为准; 未闭合细节为 Q2/Q3。

操作流图: 。

8. 正确性证明

8.1 无 false negative

命题: 无 failure 时, 已插入 key 的 Lookup 为 YES。

论证 (控制流, 已核实): 写入前缀或被更短前缀覆盖; 迁移边取边写、不丢串; Lookup 同时查旧层与新层。缺口: 对所有 n 的形式存储归纳受阶段下标冲突阻塞 (阶段下标核查项) ——不写成已完全证明。

8.2 误报率 $\leq \varepsilon$

命题: 无 failure 时, 非成员误报概率 $\leq \varepsilon$ 。骨架: 式 (2) + 阶段/键 union bound (A 精算); 迁移不重复计费。。

8.3 Prefix matching 与四结构覆盖

短串 T、长串 D; 查询固定四个活跃结构, 避免扫 $\Theta(\log n)$ 层历史 filter。

9. 时间和空间复杂度证明

9.1 时间 (B)

操作	论证	条件
Lookup	≤ 4 次底层 query + h 求值	$\neg \text{failed}$; 底层 $O(1)$
Insert	$1 \times D.\text{insert} + 10 \times \text{维护轮}$; 每轮常数底层调用	同上; 字面 10 为原文/Q3

§ 5 块重组同样去均摊到每次插入的常数步。

9.2 空间 (A 主核; B 结构约束)

Lemma 11 在 n 次插入后使用

$$n(\ell_{\lceil \log n \rceil} - \log n + O(\log \log \log u)) + u^c$$

位。代入式 (2), 并使用 $\lceil \log n \rceil - \log n = O(1)$ 、 $\log \lceil \log n \rceil = \log \log n + O(1)$, 可得

$$n(\log(1/\varepsilon) + \log \log n + O(\log \log \log u)) + u^c.$$

其中 u^c 是与输入无关的全局哈希/查表预计算空间，不得从正式定理中省略。在 Theorem 1 的 $n > u^{\{0.001\}}$ 展示条件下，选择 $c < 0.001$ 得 $u^c = o(n)$ ，且 $\log \log \log u = o(\log \log n)$ ，因而可化简为 $(1+o(1))n(\log(1/\epsilon) + \log \log n)$ 。

B 的结构核查确认，同时只有相邻两代 D/T 承担已插入前缀，而不保留 $\Theta(\log n)$ 个独立全量 filter。但四次查询本身不能证明位级峰值；初始化下一层、销毁上一层和迁移临时态的领先常数由 Lemma 11 的正式空间命题承担，本组的独立位级峰值核算仍为 $Q3$ 。

9.3 Failure $\leq \delta$ (A)

对单个已知容量结构 $D(m, \ell)$ ，第 5 节分别控制主表桶过载、fingerprint collection 超出表示预算、以及共享 $h_d(x) \cdot h_s(x)$ 的 datapoint 过多三类事件。式 (5) 将它们并合为

$$(m/\log u) \exp(-(c_3-1)^2 \log u/2) \\ + u^{\{-c_5\}} \\ + m \cdot 2^{\{-(c_4-0.01)\log u\}}.$$

因 $m < u$ ，先选择足够大的常数 c_3, c_4, c_5 ，再选满足有限独立性要求的 c_1 ，可使单个 D_i 整个生命周期的 failure 概率至多为 $u^{\{-2C\}}$ 。这是常数存在性论证，不给出未经原文支持的最小具体常数。

Claim 13 在 $n < u$ 时最多创建 $\text{ceil}(\log u)$ 个容量层，再用一次 union bound：

$$\sum_i u^{\{-2C\}} \leq (\lceil \log u \rceil) u^{\{-2C\}} \leq u^{\{-C\}} = \delta,$$

最后一步对足够大的 u 和 $C > 1$ 成立。这对应 Theorem 10 的序列级“曾报告 failure”事件。输入序列可以是任意的，概率取自预计算随机比特；本组不将原文口径无条件扩张到自适应查询对手。

10. 技术优越性

本文的非凡之处是组合保证，而不是某一个单项“首次”：

维度	本文保证	审慎评价
空间	主项含 $\log(1/\epsilon) + \log \log n$ ，另有低阶项和 u^c	在相应参数下与 PSW 的领先项对齐
时间	无 failure 时插入、查询 worst-case $O(1)$	比 PSW expected amortized 插入语义更强；不代表工程吞吐一定更快
查询结构数	只查相邻两代 D/T	避免扫描全部历史 filter
扩容	不需紧最终容量；后台迁移	filter 丢弃原键后仍能扩容是核心难点
证明	连接 FP、无 FN、failure、时间和位空间	定理强，但每项都有参数和概率前提

从技术直觉看，短串 truth table 避免把短前缀大量复制成长串；两代 D/T 让旧数据在迁移时仍可查询；每次插入推进常数轮工作，把昂贵重建去均摊。底层 $D(m, \ell)$ 再把 prefix matching 压到所需冗余。这些组件的组合，而非单独一张表或一个哈希技巧，支撑了论文的主要结果。

11. 局限性与适用范围

1. 无用户删除：Theorem 10 针对插入序列，不能称 fully dynamic。
2. 概率有两层： ϵ 是无 failure 时的误报率， $u^{\{-C\}}$ 是整段序列报告 failure 的概率；二者不是“总错误率”。
3. 有参数和预计算成本：正式定理含 $n = \omega(\log u)$ 、 $n < u$ 与输入无关的 u^c 位；非正式 $(1+o(1))$ 还需更强展示条件。
4. 固定宇宙：无需紧最终容量不等于不受宇宙 $[u]$ 限制的无限增长。
5. 实现复杂：多层哈希、adaptive prefixes、data blocks 和后台重组远比普通 Bloom filter 复杂；word-RAM 的 $O(1)$ 不能直接解释为更高实践吞吐。

6. 公开核查缺口：阶段下标形式归纳、字面常数 10 的隐藏常数配平和迁移瞬时位级峰值仍为 Q1/Q3。保留这些缺口不等于断言原论文错误。
7. 对手模型：本组不把原文保证无条件扩张到自适应查询对手。

12. 后续研究

后续工作不形成一条单轴的“替代链”。Aleph 把重点放在删除、无限增长和可运行实现；Kuszmaul-Walzer 说明删除型 dynamic filter 的额外空间下界；Resizable Retrieval 把空间参数推进到当前 n 并支持删除，但保留不同额外项和 whp 条件。由此产生四个开放方向：

1. 在保留可比的最坏时间和当前规模空间时支持删除；
2. 降低或消除 u^c 级全局随机性/预计算项；
3. 将 Claim 13 与 § 5 做成可复现实验实现，测量隐藏常数、迁移峰值与缓存行为；
4. 明确 adaptive queries/adversary 下误报与 failure 的保证。

13. 小组评价

我们认为，这篇论文最值得学习的不是某个孤立的哈希技巧，而是如何把信息论主项、当前规模空间、误报预算、无漏报、failure 概率和最坏更新时间组合起来。变长前缀解决不同插入阶段的误报预算，truth table 避免短前缀在扩展时产生过高空间开销，相邻两代 D/T 使查询在迁移期间仍只访问常数个结构，每插常数轮后台工作再将扩容去均摊。这些部件的配合，而非单个部件，构成论文的主要技术价值。

从证明表达看，论文将许多细节压缩在黑盒接口和“easy to check”式句子中，阅读门槛很高。要让结论可核验，必须把论证拆成不同责任链：式 (2) 控制误报，Claim 13 给出迁移控制流，Lemma 11 承担 prefix matching 的空间/时间/failure 接口，第 5 节控制底层坏事件，PSW 下界则只用于评价空间的不可避免性，不参与构造正确性的推导。

这篇论文的局限同样明显：它是 insertion-only 结果，依赖固定 universe、word-RAM 和 u^c 预计算，实现复杂度也远高于普通 Bloom filter。因此它的主要价值是在明确理论模型中展示一组强联合保证，而不是直接预测工程上的绝对吞吐量优势。本 Review 保留 Q1/Q3，是为了区分“原论文给出的正式命题”和“本组已经独立展开的验证”，而不是据此断言原结果错误。

14. 结论

本文不是简单的“可扩容 Bloom”，而是 unknown-size 设定下，用哈希前缀、prefix matching、短串真值表、长串 succinct 结构与分阶段去均摊迁移，逼近信息论额外 $\log \log n$ 成本并追求最坏情况常数时间。读者应同时看到定理条件、与 PSW/工程方案的模型差异，以及本文整合稿中仍开放的下标与常数配平缺口。

15. 参考文献

1. M. Liu, Y. Yin, and H. Yu. Succinct Filters for Sets of Unknown Sizes. ICALP 2020, LIPIcs 168, 79:1–79:19. DOI: 10.4230/LIPIcs.ICALP.2020.79.
2. B. H. Bloom. Space/Time Trade-offs in Hash Coding with Allowable Errors. Communications of the ACM 13(7), 1970, 422–426. DOI: 10.1145/362686.362692.
3. L. Carter, R. Floyd, J. Gill, G. Markowsky, and M. N. Wegman. Exact and Approximate Membership Testers. STOC 1978, 59–65. DOI: 10.1145/800133.804332.

4. P. S. Almeida, C. Baquero, N. Preguiça, and D. Hutchison. Scalable Bloom Filters. *Information Processing Letters* 101(6), 2007, 255–261. DOI: 10.1016/j.ipl.2006.10.007.
5. D. Guo, J. Wu, H. Chen, and X. Luo. Theory and Network Applications of Dynamic Bloom Filters. *IEEE INFOCOM* 2006, 1–12. DOI: 10.1109/INFOCOM.2006.325.
6. Y. Arbitman, M. Naor, and G. Segev. Backyard Cuckoo Hashing: Constant Worst-Case Operations with a Succinct Representation. *FOCS* 2010, 787–796.
7. E. D. Demaine, F. Meyer auf der Heide, R. Pagh, and M. Patrascu. De Dictionariis Dynamicis Pauco Spatio Utentibus. *LATIN* 2006, 349–361. DOI: 10.1007/11682462_34.
8. A. Pagh, R. Pagh, and S. S. Rao. An Optimal Bloom Filter Replacement. *SODA* 2005, 823–829. DOI: 10.1145/1070432.1070548.
9. R. Pagh, G. Segev, and U. Wieder. How to Approximate a Set Without Knowing Its Size in Advance. *FOCS* 2013, 80–89. DOI: 10.1109/FOCS.2013.17.
10. R. Raman and S. S. Rao. Succinct Dynamic Dictionaries and Trees. *ICALP* 2003, 357–368. DOI: 10.1007/3-540-45061-0_30.
11. M. A. Bender et al. Don't Thrash: How to Cache Your Hash on Flash. *PVLDB* 5(11), 2012, 1627–1637. DOI: 10.14778/2350229.2350275.
12. B. Fan, D. G. Andersen, M. Kaminsky, and M. D. Mitzenmacher. Cuckoo Filter: Practically Better Than Bloom. *CoNEXT* 2014, 75–88. DOI: 10.1145/2674005.2674994.
13. N. Dayan, I.-O. Bercea, and R. Pagh. Aleph Filter: To Infinity in Constant Time. *PVLDB* 17(11), 2024, 3644–3656. DOI: 10.14778/3681954.3682027.
14. W. Kuszmaul and S. Walzer. Space Lower Bounds for Dynamic Filters and Value-Dynamic Retrieval. *STOC* 2024, 1153–1164. DOI: 10.1145/3618260.3649649.
15. W. Kuszmaul, A. Putterman, T. Xu, H. Zhou, and R. Zhou. Resizable Retrieval. arXiv:2606.15944, 2026 预印本（截至 2026-07-28 未核实正式发表）.